

Worksheet on Creating Tables and Constraints

Create a database `customer_db` and use it to complete this worksheet.

NOT NULL constraint

Create the following table **customers** in the **customer_db**.

```
CREATE TABLE CUSTOMERS (  
    ID INT NOT NULL,  
    NAME VARCHAR (20) NOT NULL,  
    AGE INT NOT NULL,  
    ADDRESS CHAR (25),  
    SALARY DECIMAL (18, 2),  
    PRIMARY KEY (ID)  
);  
  
INSERT INTO CUSTOMERS VALUES  
(1, 'Sunath', 30, 'Colombo', 2000.00 ),  
(2, 'kamal', 28, 'Jaffna', 1500.00 ),  
(3, 'Mala', 25, 'Kandy', NULL ),  
(4, 'Manuja', 26, 'Gampaha', 6500.00 ),  
(5, 'Kathees', 22, 'Anuradhapura', 8500.00 ),  
(6, 'Anushi', 27, 'Vavuniya', NULL ),  
(7, 'Mahi', 28, 'Mahi', 10000.00 );
```

The NULLIF() Function

The SQL **NULLIF()** function compares two expressions. If both expressions are the same, it returns NULL. Otherwise, it returns the first expression. This function can be used directly with clauses like SELECT, WHERE, and GROUP BY.

```
NULLIF(expression_1, expression_2);
```

Execute the following MySQL statement and observe the result:

```
SELECT NAME, ADDRESS, NULLIF(NAME, ADDRESS) AS Result FROM  
CUSTOMERS;
```

The IFNULL() Function

This function is used to replace the NULL value with the given value.

```
IFNULL(column_name, value_to_replace);
```

Execute the following MySQL statement and observe the result:

```
SELECT NAME, SALARY, IFNULL(SALARY, 5500) AS Result FROM  
CUSTOMERS;
```

Check Constraint on Single Column

The CHECK constraint is used to enforce a condition on the values that can be inserted or updated in a column. It ensures that the data stored in the table satisfies a specified rule, thereby maintaining data integrity and validity.

If a value does not satisfy the given condition, the database rejects the insertion or update operation.

The following syntax is used to create a table with a CHECK constraint on a column.

```
CREATE TABLE table_name (  
    column_name data_type CHECK (condition)  
);
```

```
CREATE TABLE CUSTOMERS_2 (
    ID INT NOT NULL,
    NAME VARCHAR (20) NOT NULL,
    AGE INT NOT NULL CHECK (AGE>=20),
    ADDRESS CHAR (25),
    SALARY DECIMAL (18, 2),
    PRIMARY KEY (ID)
);
```

```
INSERT INTO CUSTOMERS VALUES
(1, 'Sunath', 30, 'Colombo', 2000.00 ),
(2, 'kamal', 28, 'Jaffna', 1500.00 ),
(3, 'Mala', 25, 'Kandy', NULL ),
(4, 'Manuja', 26, 'Gampaha', 6500.00 ),
(5, 'Kathees', 22, 'Anuradhapura', 8500.00 ),
(6, 'Anushi', 27, 'Vavuniya', NULL ),
(7, 'Mahi', 28, 'Mahi', 10000.00 );
```

Execute the following statement to list down the constraints implemented on CUSTOMERS_2.

```
SELECT table_name, constraint_type, constraint_name FROM
information_schema.table_constraints WHERE
table_name='CUSTOMERS_2';
```

Execute the following MySQL statement and observe the result:

```
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY) VALUES
(1, 'Mithilan', 15, 'Vanni', 2000.00 );
```

Update the age of Mathi to 18 and observe the result.

Check Constraint on Multiple Columns

```
CREATE TABLE CUSTOMERS_3 (  
    ID INT NOT NULL,  
    NAME VARCHAR (20) NOT NULL,  
    AGE INT NOT NULL CHECK (AGE >= 20),  
    ADDRESS CHAR (25),  
    SALARY DECIMAL (18, 2) CHECK (SALARY >= 20000),  
    PRIMARY KEY (ID)  
);
```

Execute the following statement to list down the constraints implemented on CUSTOMERS_3.

```
SELECT table_name, constraint_type, constraint_name FROM  
information_schema.table_constraints WHERE  
table_name='CUSTOMERS_3';
```

Execute the following MySQL statement and observe the result:

```
INSERT INTO CUSTOMERS_3 (ID,NAME,AGE,ADDRESS,SALARY) VALUES  
(1, 'Mithilan', 15, 'Vanni', 2000.00 );
```

In case if you get any error message modify the insert statement with the appropriate values and execute it again.

An alternate method to define Check Constraint

Syntax:

```
CREATE TABLE table_name (  
    column1 data_type,  
    column2 data_type,...,
```

```

CONSTRAINT constraint_name CHECK(column_name condition_value)
);

CREATE TABLE PRODUCTS (
    PID INT NOT NULL,
    PNAME VARCHAR(30),
    DELIVERY_CITY VARCHAR(20),
    DATE_OF_ORDER Date NOT NULL,
    PRICE INT,
    PRIMARY KEY(PID),
    CONSTRAINT Constraint_DOO CHECK (DATE_OF_ORDER <= '2023-
02-09')
);

```

Execute the following statement to list down the constraints implemented on CUSTOMERS_3.

```

SELECT table_name, constraint_type, constraint_name FROM
information_schema.table_constraints WHERE
table_name='PRODUCTS';

```

Execute the following MySQL statement and observe the result:

```

INSERT INTO PRODUCTS VALUES (001, 'Malik Nithin', 'Ranchi',
'2023-01-11', 2000);

INSERT INTO PRODUCTS VALUES (002, 'Mathan Ran', 'Ranchi', '2023-
03-11', 2000);

```

Check Constraint on an Existing Column

```

ALTER TABLE table_name

```

```

ADD CONSTRAINT constraint_name CHECK(ColumnName condition_value);

```

```

CREATE TABLE CUSTOMERS_4 (
    ID INT NOT NULL,
    NAME VARCHAR(20) NOT NULL,

```

```
    AGE INT NOT NULL,  
    ADDRESS CHAR (25),  
    SALARY DECIMAL (18, 2),  
    PRIMARY KEY (ID)  
);
```

Execute the following statement to list down the constraints implemented on CUSTOMERS_4.

```
SELECT table_name, constraint_type, constraint_name FROM  
information_schema.table_constraints WHERE  
table_name='CUSTOMERS_4';
```

Execute the following MySQL statement

```
ALTER TABLE CUSTOMERS_4 ADD CONSTRAINT Constraint_Age CHECK (AGE  
>= 21);
```

Execute the following statement to list down the constraints implemented on CUSTOMERS_4.

```
SELECT table_name, constraint_type, constraint_name FROM  
information_schema.table_constraints WHERE  
table_name='CUSTOMERS_4';
```

Removing a Check Constraint

```
ALTER TABLE table_name DROP CONSTRAINT constraint_name;
```

Execute the following MySQL statement

```
ALTER TABLE CUSTOMERS_4 DROP CONSTRAINT Constraint_Age;
```

Execute the following statement to list down the constraints implemented on CUSTOMERS_4.

```
SELECT table_name, constraint_type, constraint_name FROM  
information_schema.table_constraints WHERE  
table_name='CUSTOMERS_4';
```

The SQL DEFAULT Constraint

```
CREATE TABLE table_name (  
    column1 datatype DEFAULT default_value,  
    column2 datatype DEFAULT default_value,  
    column3 datatype,  
    .....  
    columnN datatype );
```

```
CREATE TABLE CUSTOMERS_5 (  
    ID INT NOT NULL,  
    NAME VARCHAR (20) NOT NULL DEFAULT 'Mithilan',  
    AGE INT NOT NULL DEFAULT '32',  
    ADDRESS CHAR (25) DEFAULT 'Colombo',  
    SALARY DECIMAL (18, 2) DEFAULT '2000',  
    PRIMARY KEY (ID)  
);
```

Execute the following MySQL statements and observe the results

```
INSERT INTO CUSTOMERS_5 (ID) VALUES (1);  
INSERT INTO CUSTOMERS_5 VALUES (2, 'Mathan', 25, 'Vavuniya',  
1500.00 );
```

```
SELECT * FROM CUSTOMERS_5;
```

Passing "DEFAULT" as Value

Execute the following MySQL statements and observe the results

```
INSERT INTO CUSTOMERS_5 VALUES  
(3, 'Kanthan', DEFAULT, DEFAULT, 2000.00),  
(4, 'Chithra', DEFAULT, DEFAULT, DEFAULT);
```

```
SELECT * FROM CUSTOMERS_5;
```

Set the default value on an existing table attribute

ALTER TABLE table_name

ALTER COLUMN column_name SET DEFAULT 'default_value';

```
CREATE TABLE BUYERS (  
    ID INT NOT NULL,  
    NAME VARCHAR (20) NOT NULL,  
    AGE INT NOT NULL,  
    ADDRESS CHAR (25),  
    SALARY DECIMAL (18, 2),  
    PRIMARY KEY (ID)  
);
```

Execute the following MySQL statements and observe the results:

```
ALTER TABLE BUYERS ALTER ADDRESS SET DEFAULT 'Delhi';  
INSERT INTO BUYERS (ID, NAME, AGE, SALARY) VALUES (01, 'Rahul',  
27, 50000);
```

```
SELECT * FROM BUYERS WHERE ID = 01;
```

Execute the following MySQL statements and observe the results:

```
ALTER TABLE CUSTOMERS_5 ALTER ADDRESS DROP DEFAULT;
```

```
DESC CUSTOMERS_5;
```